

Problem Set 04

Recursion

Computer Science & Programming (Python)
SUPSI

Instructions

- Duration: 3 hours (in-class).
- You may use your notes and slides.
- Write clear and readable code.
- Unless explicitly stated otherwise, your solutions must be **recursive**.
- When required, explain your reasoning in comments or short written answers.
- If you use helper functions, document their purpose clearly.

Exercise 1 — Tracing a Recursive Tree

Consider the following recursive function:

```
def g(n):  
    if n <= 0:  
        return 1  
    else:  
        return g(n-1) + g(n-2) + g(n-3)
```

1. Compute manually the value of $g(3)$.
2. Draw the **complete recursion tree** for the call $g(3)$.
3. Count the total number of function calls executed.
4. Determine the maximum recursion depth.
5. Based on the recursion tree, explain whether the running time is linear, polynomial, or exponential.

Exercise 2 — Recursive Function on Strings

Write a function

```
def count_vowels(s):
```

that recursively counts how many vowels appear in a string.

For this exercise:

- You must use **recursion**, not loops.
- You may assume that the input is a string.
- Consider the vowels: **a, e, i, o, u** in both lowercase and uppercase.

Example:

```
count_vowels("Recursion") -> 4
```

Then answer the following:

1. What is the base case?
2. How does the recursive call reduce the problem size?
3. What is the time complexity of your function?

Exercise 3 — Recursive Function on Lists

Write a function

```
def alternating_sum(data):
```

that recursively computes the alternating sum of a list:

$$a_0 - a_1 + a_2 - a_3 + \dots$$

For example:

```
alternating_sum([10, 3, 2, 5]) -> 10 - 3 + 2 - 5 = 4
```

Requirements:

- You must use **recursion**.
- Do not use loops.
- You may assume the list contains only numbers.

Then answer the following:

1. What is the base case?
2. What smaller subproblem is solved recursively?
3. What is the time complexity?

Exercise 4 — Recursive Construction of Substrings

Write a recursive function

```
def suffixes(s):
```

that returns the list of all suffixes of a string.

For example:

```
suffixes("code") -> ["code", "ode", "de", "e", ""]
```

Requirements:

- You must use **recursion**.
- The result must be a list of strings.
- The empty string "" must be included as the last suffix.

Then answer the following:

1. What is the base case?
2. What is the recursive case?
3. What is the time complexity with respect to the length of the string?

Exercise 5 — Recursive Class Design

In this exercise, you will design a recursive data structure.

Implement a class `NestedBox` representing a box that may contain:

- either a single integer value,
- or a list of other `NestedBox` objects.

This means that a `NestedBox` may recursively contain smaller boxes.

Part A — Constructor and representation

Implement the constructor so that the following examples are valid:

```
a = NestedBox(5)
b = NestedBox([NestedBox(1), NestedBox(2)])
c = NestedBox([NestedBox(3), b, NestedBox(4)])
```

Also implement:

- `__repr__()` for a developer-friendly representation
- `__str__()` for a readable user representation

Part B — Recursive methods

Implement the following methods recursively:

- `total()`
returns the sum of all integers stored anywhere inside the structure
- `depth()`
returns the maximum nesting depth of the structure

Examples:

- `NestedBox(5).total()` returns 5
- `NestedBox(5).depth()` returns 1
- if a box contains other boxes, its depth is 1 plus the maximum depth of its contents

Part C — Magic methods

Implement:

- `__len__()` returning the number of immediate children if the box contains a list, and 1 if it contains a single integer
- `__eq__()` so that two `NestedBox` objects are equal if their contents have the same recursive structure and values

Part D — Questions

Answer the following:

1. Why is this class naturally recursive?
2. Which methods are recursive, and why?
3. What is the time complexity of `total()` in terms of the total number of integers stored in the structure?